

Transacciones y Concurrency

Una transacción consiste en una interacción con una estructura de datos que, aun siendo compleja y estando compuesta por varios procesos que se han de aplicar uno después del otro, se quiere que sea equivalente a una interacción atómica. Es decir, que se realice de una sola vez y que la estructura a medio manipular no sea jamás alcanzable por el resto del sistema.

Ejemplo típico

Es el de la transferencia de fondos entre dos cuentas corrientes de un banco. Si queremos transferir, pongamos \$5000 de la cuenta corriente de Armando a la de Benito (A y B a partir de ahora) y las cuentas tienen, respectivamente, \$20000 y \$0 de saldo, los pasos lógicos serían:

- Comprobar si en la cuenta A hay dinero suficiente.
- Restar \$5000 de la cuenta de A, con lo que su saldo pasa a ser de \$15000.
- Sumar \$5000 a la cuenta de B, con lo que los saldos quedan $A=\$15000$ y $B=\$5000$

Ahora bien, si entre el paso 2 y el 3 el sistema sufre una parada o error inesperado las cuentas quedarían como $A=15000$ y $B=0$ con lo cual... Se han volatilizado \$5000 y presumiblemente ni Armando ni Benito estarán contentos, y hubiesen preferido que la transacción nunca hubiese sido iniciada.

Este ejemplo ilustra por qué las transacciones tienen un comportamiento deseado de Todo o nada, o se realiza completamente o no debe tener ningún efecto.

Propiedades de las transacciones

Hay cuatro propiedades importantes de las transacciones que un DBMS debe implementar para asegurar la integridad de los datos cuándo ocurren accesos concurrentes o fallas del sistema. Nos vamos a referir a estas cuatro propiedades mediante el acrónimo ACID (**A**tomicidad, **c**onsistencia, **i**solation - aislamiento - y **d**urabilidad):

- *Atomicidad*: El usuario de base de datos debe poder entender la ejecución de cada transacción cómo atómica, es decir, o todas las acciones (dentro de la transacción en cuestión) se completan exitosamente o ninguna de ellas se completa, pero nunca puede quedar un estado intermedio, de modo que el usuario no debe preocuparse por el efecto de transacciones incompletas. La base de datos garantiza ninguna transacción puede quedar "por la mitad", aunque sea removiendo o volviendo atrás los cambios que quedaron "a medio hacer", para esto, la base de datos mantiene un registro de las acciones que se registran sobre la misma, llamado Log (bitácora).

- *Consistencia*: cada transacción debe preservar la consistencia de la base de datos, su integridad. Es la propiedad por la cual una transacción siempre comienza en una instancia de base de datos consistente, por más que se encuentre (el DBMS) en "el medio" de la ejecución de otra transacción.
- *Aislamiento*: el usuario entiende que una transacción corre "sola" sin considerar el ruido que pueda hacer otra transacción ejecutándose en el mismo momento, es decir, las transacciones corren aisladas, o protegidas de los efectos de modificaciones concurrentes. La aislación de las transacciones se asegura en un DBMS de modo que ejecutar n transacciones (quizás simultáneamente) tenga el mismo efecto que ejecutar una por una de las n anteriores. Por ejemplo, existen 2 transacciones simultáneas llamadas T1 y T2, el efecto global en la base de datos de ejecutar las 2 transacciones concurrentemente es el mismo que el de ejecutar T1 seguido de T2 o T2 seguido de T1 (el motor de base de datos no asegura cuál es la transacción que se ejecuta primero, pero si asegura que existe un primero y un segundo).
- *Durabilidad*: una vez que el DBMS le confirma al usuario que su transacción ha sido exitosa, sus efectos han sido persistidos (incluso si hay una falla del sistema). El log que la base de datos mantiene de las escrituras que se hacen es utilizado también para garantizar la durabilidad de los datos, es decir para "recordar" lo que no fue persistido (quizás debido a una falla de hardware o de energía eléctrica que bajó el sistema).

Schedules

Una transacción es vista por una base de datos como una serie (lista) de acciones. Las acciones incluyen escrituras y lecturas de los datos. Una transacción puede ser definida también como un conjunto de acciones parcialmente ordenados, ya que el orden relativo de ciertas acciones puede no ser importante.

Toda transacción tiene un final claro, no siempre explícito, por ejemplo, se puede decir que la transacción termina bien y por consecuencia todos los cambios a la base de datos son finalizados, este caso se conoce como "commit". En cambio, si explícitamente o no (sea por un error en la acción a realizar programática o por una falla de hardware) se habla de un "abort" o de un "rollback".

Se define un Schedule como una lista de acciones (pueden ser lecturas, escrituras, commits o rollbacks) de un conjunto de transacciones, y el orden en el que dos acciones de una transacción T está en el schedule debe ser el mismo que el orden en el que aparece en T. Un Schedule que posee un commit o abort por cada transacción que incluye se llama Schedule Completo, que obviamente incluye todas las acciones de las transacciones en cuestión. Si el orden de ejecución de las acciones de cada transacción en el schedule es lineal, es decir, no hay acciones de distintas transacciones mezcladas, se habla de un schedule serial.

Anomalías asociadas a ejecuciones entrelazadas

Lectura sucia

Aquí una transacción T2 lee datos que todavía no han sido confirmados por una transacción T1. Nótese que una transacción debe dejar una base de datos consistente al final de su ejecución y debe encontrarla consistente antes de comenzar su ejecución, pero no se garantiza nada en el medio de la ejecución de la misma. Ver el siguiente schedule:

T1	T2
R(a)	-
W(a)	-
-	R(a)
-	W(a)
-	R(b)
-	W(b)
-	Commit
R(b)	-
W(b)	-
Commit	-

Lectura irrepetible

Suponemos que una transacción T2 puede cambiar el valor de a que ya ha sido leída por T1 mientras T1 está en progreso. Esta situación causa dos problemas:

Si T1 trata de leer el valor de a nuevamente, será distinto, lo cual no ocurriría en una ejecución serial, por eso esta situación se llama lectura irrepetible.

Suponiendo que T1 y T2 leyeron el mismo valor de a, digamos 5, y entonces T1 quiere incrementar a en 1, cambiándolo a 6, y T2 quiere decrementar a en 1, decrementa el valor que encontró (5), dejándolo en 4, corriendo estas dos transacciones en forma serial, el resultado sería siempre 5.

Overwriting Uncommitted Data

Esta situación consiste en un caso similar al siguiente: T2 pisa el valor de a, que sido ya modificado por T1 (en progreso), los problemas que puede generar se ven en el siguiente ejemplo: Suponiendo que en la tabla de empleados tenemos dos registros (representando 2 empleados) que deben mantener sus salarios iguales. T1 setea sus salarios a \$3000 y T2 los setea en \$4000, si se ejecutan transacciones seriales, o ambos quedan con un salario de \$3000 o de \$4000 (dependiendo de si fue ejecutada primero T1 o T2). Si se ejecutan en forma entrelazada, puede pasar que:

- T1 pone \$3000 a empleado1
- T2 pone \$4000 a empleado2
- T1 pone \$3000 a empleado2
- T2 pone \$4000 a empleado1

Como resultado: empleado1 tiene un salario de \$4000 y empleado2, de \$3000, lo cual es inconsistente en el modelo que imaginamos.

Ahora, extendiendo el manejo no tan sólo a las transacciones con commit sino también a las que hacen rollback (abort), la definición de schedule serial quedaría así: es un schedule cuyo efecto en cualquier instancia consistente de la base de datos garantiza ser igual a aquel cuyo schedule serial completo sobre el conjunto de las mismas transacciones excluyendo las que no hicieron commit. Por ejemplo, veamos el siguiente schedule:

T1	T2
R(a)	-
W(a)	-
-	R(a)
-	W(a)
-	R(b)
-	W(b)
-	Commit
R(b)	-
W(b)	-
Rollback	-

Nos encontramos ante un schedule irrecuperable, ya que T2 hizo commit, no podemos hacer un rollback. Se dice que un schedule es recuperable cuándo es un schedule en el que todas sus transacciones hacen commit sólo y sólo si todas las transacciones cuyos cambios leyeron hacen commit. una técnica de control de concurrencia puede prevenir estos problemas.

Strict 2 phase locking (Strict 2PL)

Si una transacción quiere leer un objeto, debe primero pedir un shared lock sobre el objeto pertinente.

Si una transacción quiere escribir un objeto, debe primero pedir un exclusive lock sobre el objeto pertinente.

Todos los locks retenidos por una transacción son liberados cuando la transacción se completa (commit o rollback)

Cumpliendo estas reglas, un DBMS puede entrelazar acciones entre transacciones de forma segura.

Supongamos la existencia de las siguientes transacciones con la siguiente lista de acciones:

- T1
 - o R(a)
 - o R(c)
 - o W(c)
- T2
 - R(a)
 - R(b)
 - W(b)

Un schedule posible para una ejecución serializada es:

T1	T2
R(a)	-
R(c)	-
W(c)	-
Commit	-
-	R(a)
-	R(b)
-	W(b)
-	Commit

Y un schedule para una ejecución entrelazada posible sería:

T1	T2
S(a)	-
R(a)	-
-	S(a)

-	R(a)
-	X(b)
-	R(b)
-	W(b)
-	Commit
X(c)	-
R(c)	-
W(c)	-
Commit	-

Como conclusión, es evidente que el strict 2 phase locking ayuda a administrar transacciones a un DMBS pero no alcanza para evitar deadlocks, para eso existen técnicas de control de concurrencia.

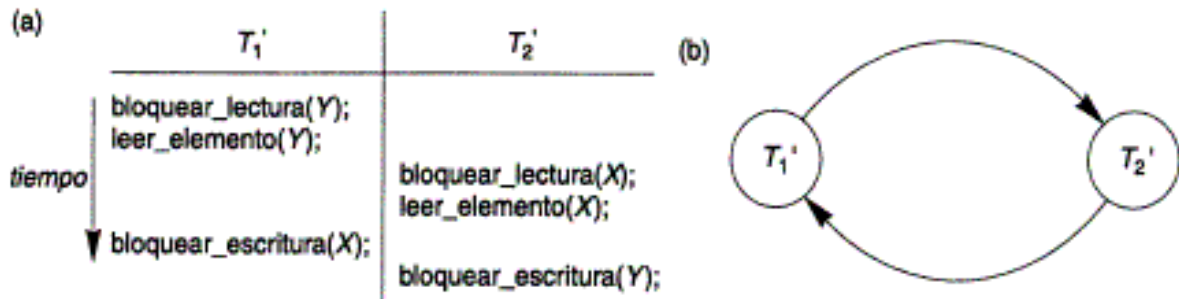
Una variante conservadora del S2PL consiste en que cada transacción, antes de comenzar con las operaciones propiamente dicha, pida al DBMS todos los candados necesarios. De ese modo pueden evitarse deadlocks.

Deadlock

Ocurre deadlock cuándo dos transacciones simultáneas están esperando que la otra libere su candado sobre un elemento. En el siguiente schedule o plan, se muestra un ejemplo sencillo:

T1	T2
S(a)	S(b)
R(a)	R(b)
X(b)	X(a)
Free(a)	Free(b)
R(b)	R(a)
W(b)	W(a)
Commit	Commit

Una posibilidad para evitar el deadlock es utilizar la variante conservadora del S2PL, pero evidentemente se estaría limitando la concurrencia.



El problema del bloqueo mortal. (a) Plan parcial de T_1' y T_2' que está en estado de bloqueo mortal. (b) Grafo de espera del plan parcial

Otro protocolo de prevención de deadlock que han utilizado implementaciones arcaicas, consiste en ordenar los recursos de la base de datos de forma total y hacer uso dentro de las transacciones en ese orden, obligando al programador a conocer ese orden... La verdad poco científico y poco práctico...

Otra alternativa consiste en etiquetar con timestamps las transacciones. Suponiendo que hay 2 transacciones que esperan por recursos cruzados (cómo en la figura arriba), las reglas que se pueden utilizar son:

- *esperar-morir*: Una transacción más antigua puede esperar a que termine una más nueva, pero una transacción más reciente que solicite un elemento bloqueado por una más antigua se abortará y reiniciará.
- *herir-esperar*: Se hace lo contrario, una transacción más reciente puede esperar a que termine una más antigua, pero una transacción más antigua que solicite un elemento bloqueado por la otra, desalojará a la más reciente abortándola.

En última instancia, las dos reglas hacen lo mismo: que aborté la más reciente. Si bien se puede demostrar que estas dos técnicas son libres de deadlocks, sin embargo estas técnicas hacen que muchas transacciones se aborten y reinicien aunque no necesariamente vayan a causar deadlocks.

Otras técnicas de concurrencia no necesitan utilizar timestamps, como por ejemplo:

- *No Waiting (NW)*: Si una transacción no puede obtener un candado, se aborta de inmediato y se reinicia después de cierto tiempo aleatorio sin comprobar si ocurrirá o no un deadlock. Obviamente este enfoque puede hacer que muchas transacciones se aborten innecesariamente.
- *Cautious Waiting (CW)*: Es una variación de NW, pero que intenta evitar la cantidad de transacciones abortadas innecesariamente. Suponiendo que la transacción T_i intenta bloquear un elemento X , pero no puede, dado que el elemento X está siendo bloqueado por una transacción T_j con un candado en conflicto. Las reglas son las siguientes:

- Si T_j no está detenida (esperando un elemento bloqueado por otra transacción), entonces se detiene T_i , la cual puede esperar, de lo contrario se aborta T_i .

Puede demostrarse que CW es libre de deadlocks, considerando los momentos en que una transacción T queda detenida $d(t)$. Si las dos transacciones T_i y T_j quedan detenidas, y T_i está esperando que T_j termine, entonces $d(T_i) < d(T_j)$, ya que una transacción sólo puede esperar a que termine otra cuando no está detenida. Por tanto, los momentos de detención constituyen un ordenamiento total sobre todas las transacciones detenidas, y no puede ocurrir ningún ciclo que provoque un deadlock.

Otra técnica consiste en utilizar tiempos predefinidos a fin de abortar las transacciones que esperen por un bloqueo de otra transacción más de ese tiempo pre estipulado (time out).

Otro enfoque distinto, consiste en detectar los deadlocks, en la cual el DBMS verifica periódicamente si el motor está en un estado de deadlock. Este enfoque obviamente no es muy útil cuando la carga transaccional del DBMS es muy alta o la longitud de las transacciones es muy larga. No es recomendado para sistemas OLTP. Una forma sencilla de detectarlos es construyendo un grafo de espera:

- Se crea un nodo por cada transacción del schedule (plan)
- Se crea un arco de T_i a T_j si la T_i está esperando para bloquear un elemento X que está bloqueado momentáneamente por T_j .
- Se elimina el arco anterior cuando T_j libera el/los candado/s sobre el/los elementos que estaba esperando T_i .
- Si en el grafo hay un ciclo, quiere decir que hay un estado de deadlock.

El problema de este enfoque consiste en determinar cuándo debe el DBMS determinar que hay o no deadlock. Si el sistema determina que hay deadlock, determinará que transacción o que transacciones abortará y reiniciará, a estas transacciones se las llama *transacciones víctimas*. Aun así existen problemas que pueden ocurrir:

El llamado **reinicio cíclico**, que consiste en que una transacción se reinicia indefinidamente para entrar en deadlock indefinidamente.

La **espera indefinida**, que se da cuando una transacción no puede continuar durante un período indeterminado mientras otras transacciones pueden continuar. Generalmente esto ocurre cuando el esquema de espera es injusto, confiriendo prioridades muy bajas a la transacción que cae en este estado. Una solución a este problema consiste en utilizar esquemas de espera más justos, generalmente implementados vía una cola FIFO dando prioridades de esta manera en el esquema de bloqueo.

La **inanición** ocurre cuando la transacción víctima es siempre la misma.

Control de concurrencia basado en ordenamiento por TimeStamp

Se asigna un timestamp a cada transacción dentro de un plan o schedule. El algoritmo debe asegurar que por cada elemento utilizado por más de una transacción en el plan, el orden en que se tiene acceso al elemento no viole la serializabilidad del schedule, para lograr esto, el algoritmo básico va a asociar a cada elemento X dos timestamp:

- $ReadTS(X)$: consiste en el timestamp más grande de todos los timestamp de las transacciones que han leído con éxito el elemento X.
- $WriteTS(X)$: consiste en el timestamp más grande de todos los timestamp de las transacciones que han escrito con éxito el elemento X.

Siempre que una transacción intente leer o escribir el elemento X, el algoritmo básico compara el timestamp de la transacción con el $ReadTS$ y el $WriteTS$ de X, para asegurar que no se viole el orden de ejecución. Si la transacción viola el orden, también viola el plan en serie equivalente, así que se aborta, reintroduciéndose al sistema con un timestamp más nuevo. Ahora hay que pensar que estamos dentro de un schedule y que el hecho de que se aborte una transacción cualquiera puede hacer abortar una transacción anidada, lo cual se conoce como reversión anidada (planes no recuperables) y es un problema asociado con este algoritmo básico. Es por eso que el algoritmo debería verificar si se viola el ordenamiento por TimeStamp de las transacciones estos casos:

- La transacción T emite una operación $w(X)$:
 - o Si $ReadTS(X) > TS(T)$ o $WriteTS(X) > TS(T) \Rightarrow$ abortar y revertir T y rechazar operación, ya que alguna transacción con timestamp mayor que el de T (por tanto posterior) ya leyó o escribió el valor del elemento X antes de que T tuviera oportunidad de escribir X, violándose así el ordenamiento por timestamp.
 - o Si no se presenta la situación anterior, se ejecuta la operación $w(X)$ de T y se le asigna $TS(T)$ a $WriteTS(X)$.
- La transacción T emite una operación $r(X)$:
 - o Si $ReadTS(X) > TS(T)$ o $WriteTS(X) > TS(T) \Rightarrow$ abortar y revertir T y rechazar operación, ya que alguna transacción con timestamp mayor que el de T (por tanto posterior) ya escribió el valor del elemento X antes de que T tuviera oportunidad de leer X.
 - o Si $WriteTS(X) \leq TS(T)$, se ejecuta la operación $r(X)$ de T y se le asigna a $ReadTS(X)$ el mayor de los dos valores $TS(T)$ y $ReadTS(X)$ actual.

De esta manera, el algoritmo comprueba si dos operaciones en conflicto ocurren en el orden incorrecto, y rechaza la más reciente abortando la transacción que la emitió. De esta manera se garantiza que los schedules involucrados son seriables por conflictos. Hay una modificación del algoritmo llamada **regla de escritura de Thomas** que no impone la seriabilidad por conflictos, pero rechaza menos operaciones de escritura, modificando las verificaciones de la operación $w(X)$ de la siguiente manera:

- Si $ReadTS(X) > TS(T) \Rightarrow$ abortar y revertir T y rechazar la operación
- Si $WriteTS(X) > TS(T) \Rightarrow$ no ejecuta la operación de escritura, pero continúa procesando.
- Sino, se ejecuta la operación $w(X)$ de T y se asigna $TS(T)$ a $WriteTS(X)$.

Como conclusión sobre este algoritmo podríamos decir que:

- Garantiza la seriabilidad de los planes (schedules)

- Algunos planes son posibles con algunos de estos protocolos pero no con otros
- No hay deadlocks
- Puede haber reinicio cíclico
- Puede haber inanición

Técnicas de Multiversión

Hay una serie de técnicas que guardan las versiones anteriores de los datos (en cuanto a las transacciones que los afectan), esto implica consumir mucho más recursos de almacenamiento. Dentro de esta familia de técnicas, la variedad es enorme, y comercialmente muchos motores implementan variaciones y combinaciones de la misma (cabe destacar el caso de los segmentos de rollback de Oracle).

Procesamiento de Transacciones Distribuidas

Las bases de datos distribuidas surgieron cuando surgió la necesidad o la imposibilidad de administrar grandes cúmulos de datos de manera centralizada. Sea esta la razón, o sea porque la naturaleza de las bases de datos es diferente¹, surgió casi en paralelo la necesidad de poder coordinar transacciones de manera distribuida.

El caso de las bases de datos distribuidas es más complejo, pues deben estudiarse protocolos de replicación distribuida, protocolos de confiabilidad distribuidos, control de concurrencia distribuido y el procesamiento y ejecución de consultas distribuidas. En este apartado nos vamos a concentrar en el manejo de transacciones distribuidas, que es sólo uno de los temas técnicos a estudiar dentro de la temática de bases de datos distribuidas.

La gestión de transacciones distribuidas va a centrarse en la problemática de cómo una entidad llamada *"Coordinador de Transacciones Distribuidas"* va a poder coordinar transacciones entre distintas instalaciones de bases de datos, como si se trataran de transacciones ejecutadas en una sola base de datos, garantizando las propiedades ACID. Por tanto el procesamiento de transacciones distribuidas es una forma de OLTP² que permite que una transacción esté compuesta por operaciones de múltiples aplicaciones que acceden a una³ o más bases de datos (en uno o en más equipos a través de una red). Por tanto este tipo de transacciones son llamadas transacciones distribuidas.

Para comenzar con un ejemplo, vamos a volver al caso del principio del documento, en el cual vimos que se intentaba realizar una transferencia de fondos entre dos cuentas corrientes de un banco. Vamos a suponer por un instante (aunque quizás no tenga sentido práctico) que una de las cuentas se encuentra en otra base de datos (incluso gestionada por otro RDBMS en una ubicación diferente a la primer cuenta). Si queremos transferir, pongamos \$5000 de la cuenta corriente de Armando a la de Benito (A y B a partir de ahora) y las cuentas tienen, respectivamente, \$20000 y \$0 de saldo, los pasos lógicos serían:

¹ Ya sea técnicamente (fabricante del DBMS, plataforma o sistema operativo) o funcional (bases de datos totalmente distintas, en empresas físicamente separadas, etc).

² Online Transaction Processing.

³ El caso en el que varias aplicaciones en distintos contenedores pueden transaccionar contra una misma base de datos también es un caso de procesamiento de transacciones distribuido.

- Comprobar si en la cuenta A (DBMS1) hay dinero suficiente.
- Restar \$5000 de la cuenta de A (DBMS1), con lo que su saldo pasa a ser de \$15000.
- Sumar \$5000 a la cuenta de B (DBMS2), con lo que los saldos quedan A=\$15000 y B=\$5000

Ahora bien, si en el paso 3 el sistema (DBMS2) sufre una parada o error inesperado las cuentas quedarían como A=15000 y B=0 con lo cual... Se han volatilizado \$5000 y presumiblemente ni Armando ni Benito estarán contentos, y hubiesen preferido que la transacción nunca hubiese sido iniciada. Es por eso que toda la transacción (la transacción global) debe respetar las propiedades ACID.

Componentes para el Procesamiento de Transacciones Distribuidas

Hay un estándar abierto para el procesamiento de transacciones distribuidas, propuesto por un consorcio de propietarios de software (Open Group), llamado X/Open XA (X Open Distributed Transaction Processing Model). Dentro de ese modelo, los componentes son:

- Application Program: La lógica de negocio desarrollada por los programadores de las aplicaciones para cumplir una o más tareas específicas, teniendo en cuenta que quizás se trate de más de una aplicación a la vez.
- Transaction Manager: Software que coordina las transacciones globales y asegura la sincronización de las bases de datos en los casos en que sea necesario.
- Resource Manager: Software que administra parte de los recursos compartidos del sistema, tales como las bases de datos. Estos recursos no sólo pueden estar representados por DBMS, sino que pueden ser gestores de colas (JMS por ejemplo), middleware de comunicaciones, y otras piezas de software.
- Communications Resource Manager: Software que sirve como un Resource Manager especializado para comunicar datos entre las aplicaciones.

Las tecnologías más importantes de la industria adoptaron este modelo, como por ejemplo EJB en Java y MTS en ambientes Microsoft.

Los roles que se ven durante la ejecución de una transacción distribuida son:

- Coordinador: Es el Transaction Manager que administra la transacción global.
- Participante: Es uno de los varios Resource Managers (DBMS generalmente) que son afectados por la transacción distribuida.

El modelo X/Open XA define una API de DTP para ser empleadas por las aplicaciones y los Transaction managers con los siguientes componentes:

- AP-RM: Consiste en la interfaz entre las aplicaciones y el Transaction Manager que permite que un recurso (una DBMS) sea actualizado (o modificado). La interfaz en sí, no está explícitamente especificada, una instancia de este componente es el conocido SQL.
- TX: La interfaz entre la aplicación y el Transaction Manager que permite que la aplicación arranque transacciones globales.

- CM: La interfaz entre la aplicación y el communications resource manager que permite que la aplicación se comunique con otras aplicaciones dentro de la API de transacciones distribuidas. Son ejemplos la interfaz cliente/servidor, la interfaz peer-to-peer y la interfaz RPC (remote procedure call).
- System Interfaces: El modelo especifica estas interfaces de sistema que permiten a los componentes del mismo interactuar con los demás. Cabe destacar que estas interfaces son casi siempre invisibles para los programadores de aplicaciones.
- XA: La interfaz entre el resource manager y el transaction manager que permite que coordinen transacciones globales entre ellos. Esta interfaz habilita la posibilidad de que operen con un protocolo de commit en dos fases, lo que finalmente asegura la sincronización de las bases de datos.
- XA+: La interfaz entre el transaction manager y el communications resource managers que les permite coordinar transacciones entre computadoras en un sistema DTP⁴.

Commit en dos fases

Este algoritmo permite que todos los nodos de un sistema distribuido puedan ponerse de acuerdo para confirmar una transacción distribuida. Este algoritmo está orientado a garantizar las propiedades ACID en un DTP, y que todos los participantes estén sincronizados.

El Transaction manager asociado con la aplicación que inicia la transacción será llamado el coordinador y las bases de datos involucradas dentro de la transacción global serán llamados los participantes. En Commit en Dos Fases, el coordinador orquesta la sincronización de todos los participantes, para eso implementa dos fases:

- Primera fase: Fase de Petición del Commit, en donde el coordinador pregunta a todos los participantes si pueden garantizar un commit y los participantes responden. Todos los participantes deben confirmar su actuación.
- Segunda fase: Fase de Commit, en donde el coordinador instruye a todos los participantes a hacer efectivo el commit pedido den la fase anterior (si todos estaban listos), o de abortar la transacción (si alguno de los participantes no estaba listo). Todos los participantes deben confirmar su actuación.

El coordinador debe asignar un identificador de transacción global (GTRID) para identificar unívocamente a la transacción global (sin confundirla con las transacciones locales que se pueden dar en los resource managers) cuando una aplicación lanza una transacción.

Cada vez que la aplicación accede a través del Transaction manager a un resource manager involucrado en la transacción en la que se está trabajando, el coordinador etiqueta estos mensajes con el GTRID correspondiente para que los resource manager puedan monitorear independientemente qué registros son accedidos por la transacción global.

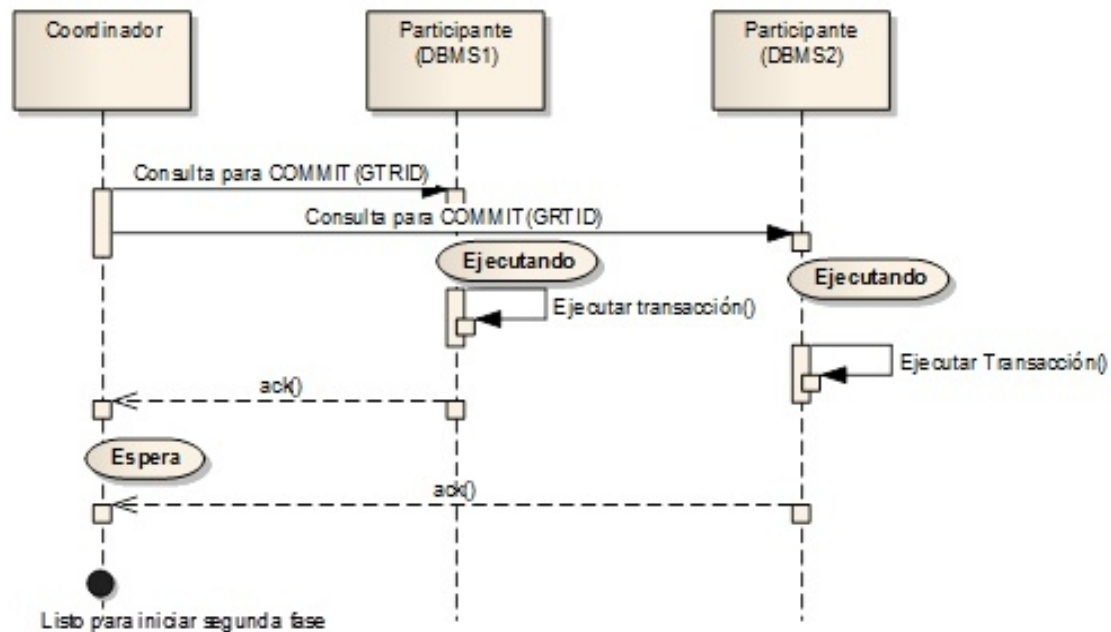
Cuando la aplicación le indica al Transaction manager que la transacción puede ser confirmada, el coordinador realiza un protocolo de commit en dos fases, específicamente:

- Contacta a todos los resource managers involucrados en la fase primera.

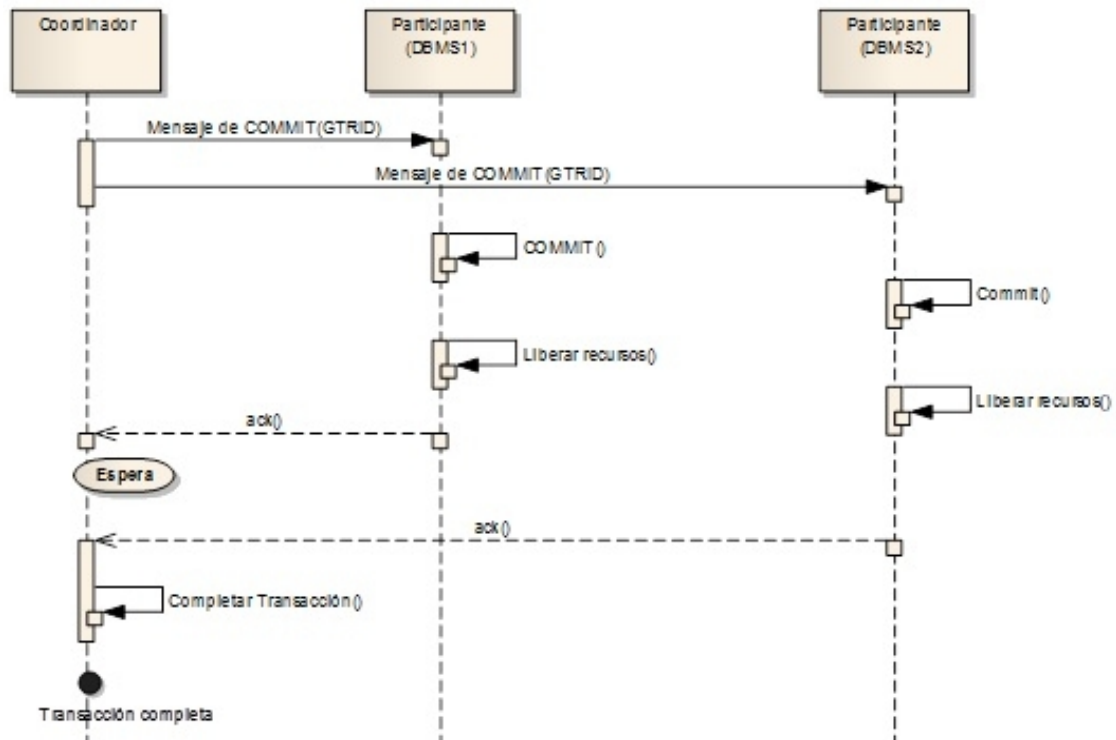
⁴ Distributed Transaction Processing

- Lleva log de las salidas o respuestas de la fase primera.
- Contacta a todos los resource managers involucrados en la segunda fase.

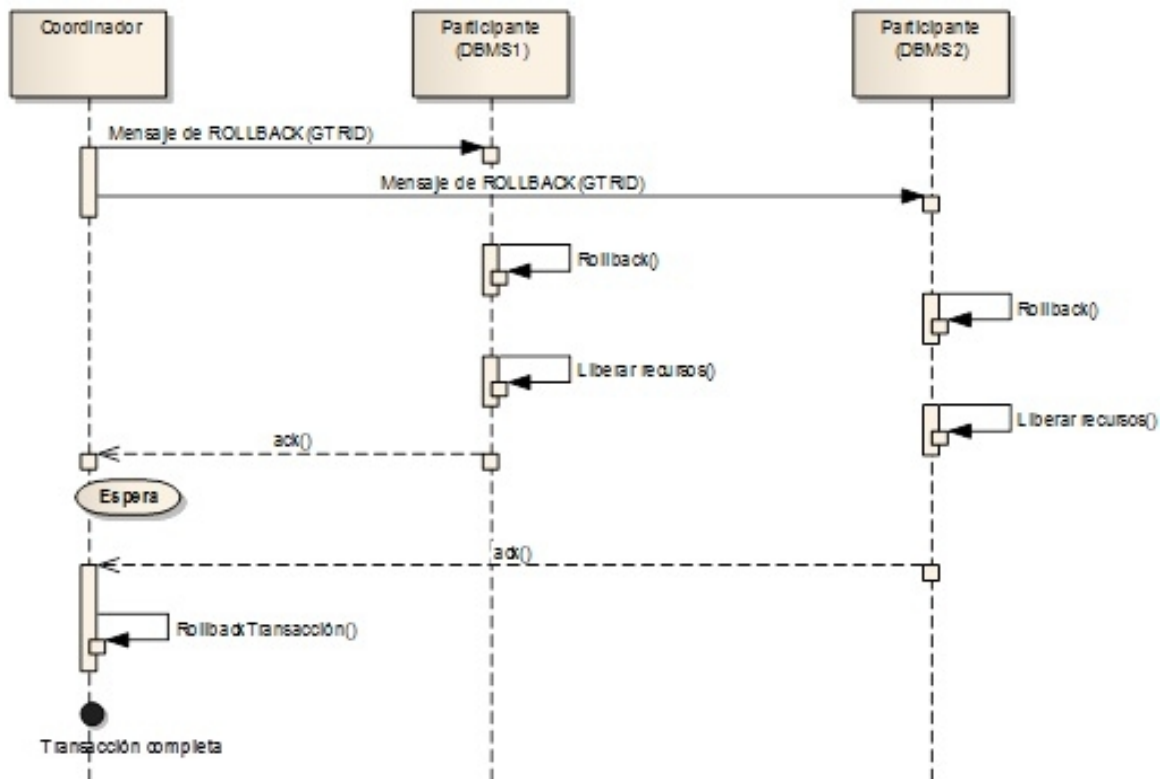
En el siguiente diagrama se puede ver la secuencia de acciones durante la primera fase del algoritmo:



En el caso en el que todos los participantes devolvieron un mensaje de ack indicando el éxito de la transacción, la segunda fase se describe en el siguiente diagrama de secuencia:

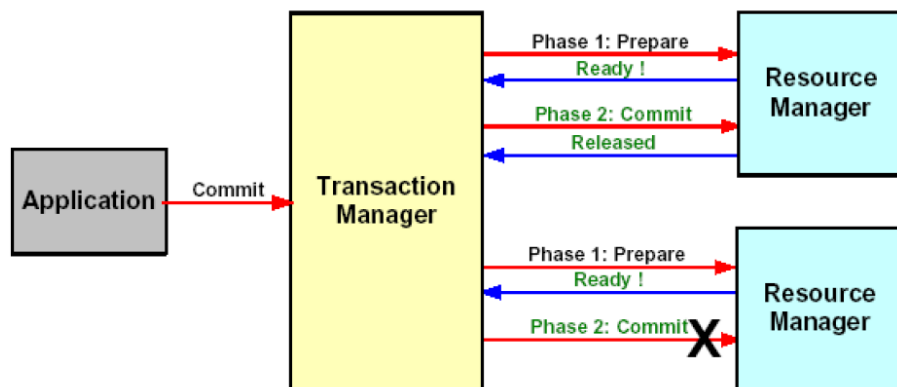


En el caso en el que alguno de los participantes devolviera un mensaje de ack indicando abortar la transacción, la segunda fase se describe en el siguiente diagrama de secuencia:



El coordinador registra en un log propio (TLog) el estado de la transacción global, de manera similar a como lo hace un RDBMS cuando procesa una transacción local. Al final de la primera fase del commit en dos fases, el coordinador registra en el TLog los resultados que le enviaron todos los participantes de la transacción global. De esa manera puede mantener un estado global de lo sucedido.

Existen ciertos resquicios que la implementación de los protocolos deben evitar, por ejemplo si en la segunda fase del commit la confirmación de una de las partes no llega nunca, el coordinador indicará después de un tiempo que se revierta la transacción iniciando un protocolo de recuperación, pero si la falla es más de tipo permanente, no se estará seguro de cómo se iniciará el estado del resource manager caído. Allí surgen las transacciones “en duda” ya que el commit pudo haber sido hecho y nunca haber llegado el ack (falla de comunicación), como se ilustra en el siguiente diagrama:



La mayor desventaja del protocolo commit en dos fases es el hecho de que es un protocolo bloqueante. Un nodo estará bloqueado mientras esté esperando un mensaje. Esto significa que otros procesos compiten por los bloqueos de recursos mantenidos por los procesos bloqueados tendrán que esperar a que los bloqueos sean liberados. Un nodo individual continuará incluso si otros resource managers han fallado. Si el coordinador falla permanentemente, algunos participantes nunca resolverán sus transacciones. Esto tiene el efecto de que los recursos están siempre comprometidos. El algoritmo puede bloquear indefinidamente de la siguiente forma: si un participante ha enviado un mensaje acuerdo al coordinador, se bloqueará hasta que un commit o rollback sea recibido. Si el coordinador está permanentemente caído, el participante bloqueará indefinidamente, a menos que pueda obtener la decisión global commit/abort desde uno de los participantes. Cuando el coordinador ha enviado "consulta para commit" a los participantes, esto bloqueará hasta que todos los participantes hayan enviado su decisión local. Más aún, si un participante está permanentemente caído, el coordinador no bloqueará indefinidamente: Dado que el coordinador es el único en decidir si la decisión es 'commit' o 'abort' el bloqueo permanente puede evitarse introduciendo un timeout: Si el coordinador no ha recibido todos los mensajes esperados cuando ha pasado el timeout decidirá 'abortar'. Este comportamiento conservador del protocolo es otra desventaja: Se predispone al caso de aborto en lugar del caso completo.

Protocolo commit de dos fases en árbol

Una variante común del protocolo de commit en dos fases en un sistema distribuido es el Protocolo de commit de dos fases en árbol. En esta variante el coordinador es la raíz ("top") de un árbol de comunicaciones, mientras los participantes son los otros nodos. Los mensajes desde el coordinador son propagados "hacia abajo" en el árbol, mientras los mensajes al coordinador son "recolectados" por un participante desde todos los participantes que hay debajo, antes de que esto envíe el mensaje apropiado "hacia arriba" por el árbol (excepto con un mensaje "abortar", el cual es propagado "hacia arriba" inmediatamente hasta que se reciba, o si este participante decide abortar).

El protocolo commit de dos fases dinámico (Dynamic two-phase commitment, D2PC) es una variante del protocolo de commit de dos fases en árbol sin coordinador predeterminado. Los ack comienzan propagándose a todas las hojas, cada hoja cuando completa sus tareas de acuerdo con la transacción, y el coordinador es determinado dinámicamente por los mensajes ack en carrera, en los lugares donde se concentran. Ellos se concentran en un nodo árbol de la transacción, o en un canal. En el último caso uno de los nodos que están al extremo del cable es elegido como un coordinador (cualquier nodo). D2PC es óptimo respecto al tiempo, permitiendo la liberación temprana de los recursos bloqueados.

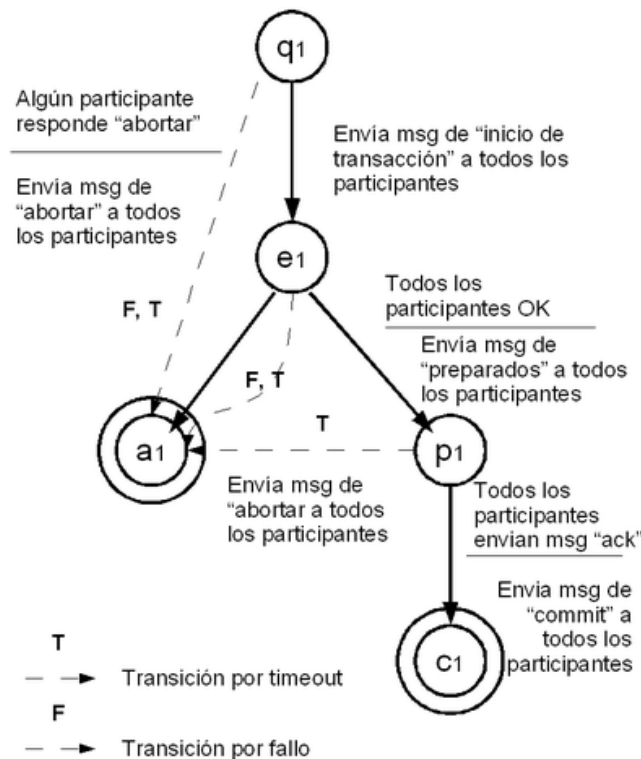
Protocolo commit de tres fases

Este protocolo fue diseñado para evitar el bloqueo que ocurre en el algoritmo de commit de dos fases. Específicamente, 3PC sitúa un límite superior de la cantidad de tiempo requerido antes de que una transacción haga el commit o abort. Esta propiedad asegura que si una transacción dada está intentando hacer commit vía 3PC y mantiene algún recurso bloqueado (locking), puede liberar los bloqueos después del límite de tiempo (timeout).

Las tareas realizadas por el coordinador son las siguientes:

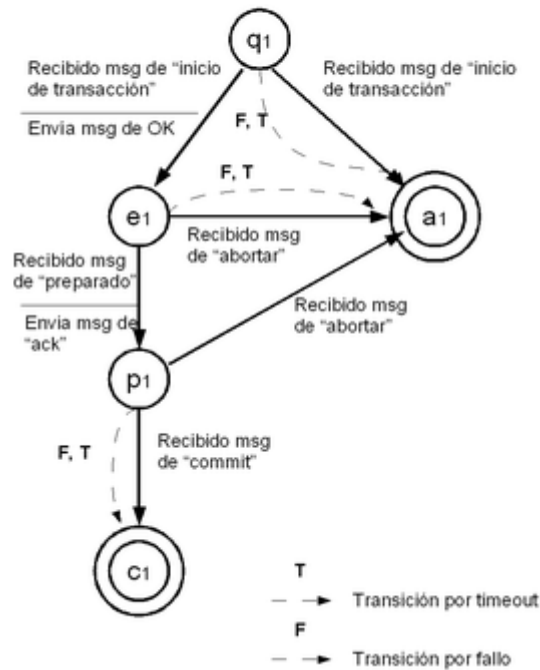
- El coordinador recibe una solicitud de transacción. Si hay un fallo en este momento, el coordinador aborta la transacción (en cuanto se recupere hace la transición por fallo). En caso contrario, el coordinador envía un mensaje de inicio de transacción a los participantes y cambia al estado de en espera.

- Si hay un fallo, timeout, o si el coordinador recibe un mensaje de no se iniciará la transacción en el estado de en espera, el coordinador aborta la transacción y envía un mensaje de abortar a todos los participantes. En caso contrario el coordinador recibirá mensajes de puede comenzar la transacción desde todos los participantes dentro de la ventana de tiempo, y por tanto envía mensajes de commit a todos los participantes y cambia al estado de preparados.
- Si el coordinador falla en el estado de preparados, cambiará al estado de commit. Sin embargo si el coordinador se pasa de tiempo mientras espera un ack de un participante, abortará la transacción. En el caso de que todos los ack son recibidos, el coordinador cambia también al estado de commit.



Las tareas realizadas por los participantes son las siguientes:

- El participante recibe un mensaje de inicio de transacción desde el coordinador. Si el participante está de acuerdo, contesta con un mensaje indicando que se iniciará la transacción (OK) al coordinador, y cambia al estado de en espera. En caso contrario envía un mensaje indicando que no se iniciará la transacción y aborta. Si hay un fallo, cambia al estado de abortar.
- En el estado de en espera, si el participante recibe un mensaje de abortar desde el coordinador, tiene un fallo, o se pasa de tiempo esperando un commit, aborta. Si el participante recibe un mensaje de preparado, contesta con un ack y pasa al estado de pendiente.
- Una vez en el estado de pendiente, al recibir un mensaje de commit, confirma la transacción realizando el commit. En caso de fallo o de timeout también hace commit.



La principal desventaja de este algoritmo es que no puede recuperarse en el caso de que la red sea segmentada de alguna forma.

Recuperación

Con el fin de garantizar las propiedades ACID previamente enunciadas, el DBMS necesitará procurarse de un mecanismo de recuperación. La naturaleza de las excepciones que debe tratar se pueden resumir de la siguiente manera:

- Falla de energía o de memoria
- Error del sistema operativo
- Excepciones controladas por la aplicación o por los objetos del DBMS que retraen una transacción (rollback)
- Imposición del Transaction Manager (por ejemplo, el control de deadlocks decide interrumpir una transacción y lanzarla en un intervalo aleatorio posterior de tiempo)
- Falla de disco
- Problemas y Catástrofes físicas

Log (Bitácora) de Transacciones

Con el fin de recuperarse ante las fallas anteriormente citadas, los DBMS suelen mantener un log transaccional que le sigue la pista a todas las operaciones que se realizan. Suele mantenerse en disco y se recomienda su respaldo para protegerlo de los fallos catastróficos.

En el log, las entradas las maneja cada sistema, pero esencialmente habrá entradas de inicio de transacción, de fin de las mismas (positivos o negativos), de escritura de recursos, y de un tipo especial, llamado checkpoint.

Este checkpoint le indica al sistema el momento hasta el cuál se han volcado a los archivos físicos de datos las entradas del log.

Proceso de Recuperación

Las veces que el DBMS va a necesitar recurrir a los backups físicos (totales e/o incrementales) será porque el tipo de falla es más bien catastrófica. De otra manera, le alcanzará con acceder al log, a fin de deshacer (o quizás rehacer) alguna transacción en curso al momento de la falla.

Técnicas de recuperación basadas en actualización diferida

Se conoce como actualización diferida el mecanismo por el cual, al interactuar transaccionalmente con un DBMS no se graban los datos directamente al almacenamiento físico, sino que se escribe en un log al momento de la confirmación de la transacción, y luego de manera diferida a los archivos físicos de datos al momento del checkpoint del log transaccional.

En estos casos, si una transacción falla antes de llegar a su confirmación (commit), no hay necesidad de deshacer ninguna operación, dado que la transacción no ha afectado la base de datos en ningún modo.

Hay ciertas premisas que siguen estas técnicas, que pueden resumirse de la siguiente manera:

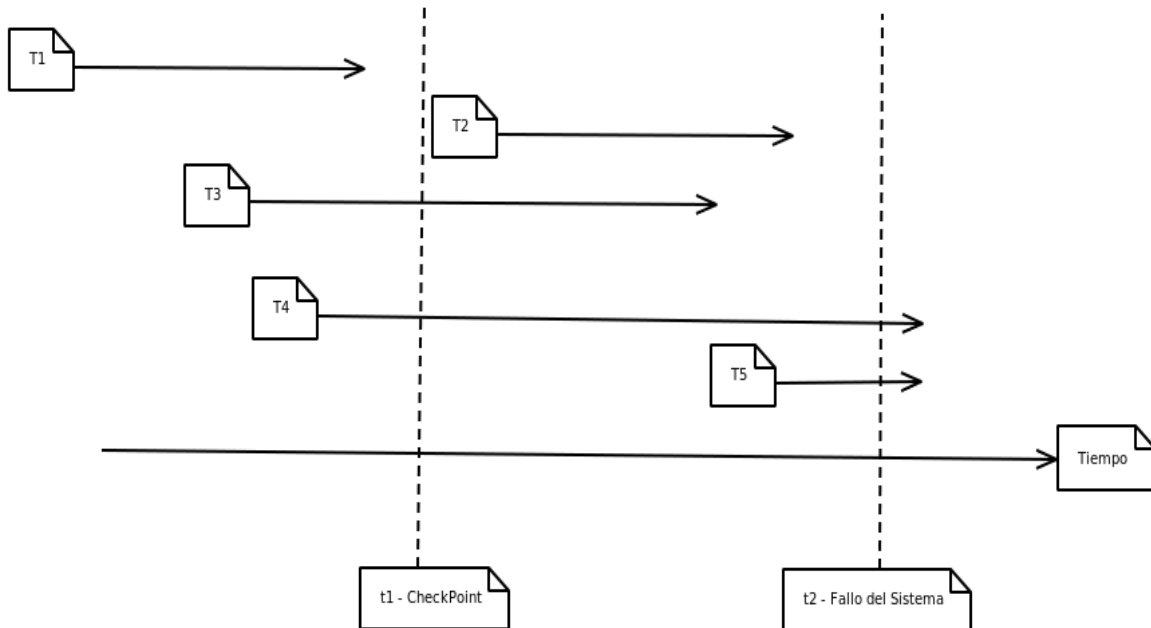
- Una transacción no puede modificar la base de datos antes de llegar a su punto de confirmación (commit)
- Una transacción no llega a su punto de confirmación sin antes asentar todas sus operaciones de actualización al log transaccional y forzar la escritura (flush) del log al medio (disco).

El procedimiento para la recuperación, puede describirse de la siguiente manera:

- El DBMS mantiene dos listas de transacciones:
 - Las transacciones T confirmadas desde el último checkpoint
 - Las transacciones activas Ta
- Rehacer, a partir del log, todas las operaciones (de escritura) de transacciones confirmadas en el orden en que aparecen en el log.

- Las Ta que no han sido confirmadas se cancelan efectivamente y deberán reintroducirse.

Por ejemplo, para el siguiente Schedule:



Según el procedimiento anteriormente descrito, la transacción T1 se había confirmado antes del último checkpoint (antes de la falla), por lo que no necesita ninguna acción en particular, ya fue escrita a la base de datos. Las transacciones T2 y T3 deben rehacerse, dado que no fueron escritas definitivamente a la base de datos, por lo haber verificado un checkpoint. Las transacciones T4 y T5 se ignoran, nunca recibieron una confirmación.

Una variante para optimizar este procedimiento es verificar las veces que se escribe un mismo recurso y modificarlo sólo la última vez, o mejor aún, procesar de atrás hacia adelante omitiendo las escrituras previas del mismo recurso.

Técnicas de recuperación basadas en actualización inmediata

Se conoce como actualización inmediata el mecanismo por el cual, al interactuar transaccionalmente con un DBMS se graban los datos directamente e inmediatamente al almacenamiento físico. Sin embargo, también se utiliza un log transaccional, a fin de que el DBMS sea capaz de recuperarse en caso de fallos. Estas variantes hacen uso de un mecanismo de escritura anticipada en el log transaccional.

En estos casos, las medidas a tomar ante una recuperación tienen que ver con deshacer las escrituras realizadas en la base de datos.

En estas técnicas, también se asientan checkpoints en los logs, y el procedimiento puede describirse de la siguiente manera:

- El DBMS mantiene dos listas de transacciones:
 - Las transacciones confirmadas desde el último checkpoint (T_c)
 - Transacciones activas (T_a)
- Deshacer todas las operaciones de T_a en el orden opuesto al que fueron asentadas en la bitácora.
- Rehacer todas las operaciones de T_c en el orden en que se escribieron en la bitácora.

Técnicas de recuperación con Paginación de Sombra

Esta técnica concibe que la base de datos esté compuesta por varios bloques (páginas) de disco de tamaño fijo para fines de recuperación. En esa línea, se constituye una tabla de páginas con n entradas donde la i -ésima entrada a la misma apunta a la i -ésima página de la base de datos en el disco. Esta tabla se mantiene en memoria (si no es demasiado grande) y todas las referencias a las páginas de la base de datos en el disco pasan por la tabla de páginas. Cuando una transacción comienza a ejecutarse, la tabla de páginas actual se copia a una tabla de páginas sombra, que se guarda en disco mientras la transacción usa la tabla actual.

Durante la ejecución de la transacción, la tabla de páginas sombra nunca se modifica, cuando se efectúa una operación de escritura, se crea una nueva copia de la página modificada, pero nunca se sobrescribe la copia antigua. En vez de ello, la nueva página se escribe en otro sitio.

En el caso de las páginas actualizadas por las transacciones, se mantienen dos versiones. La tabla de páginas sombra hace referencia a la versión antigua, y la tabla de páginas actuales a la nueva.

La recuperación de una falla mientras se ejecuta una transacción es trivial: Se liberan las páginas modificadas de la base de datos y se deshace la tabla de páginas actuales. El estado consistente anterior está disponible a través de este mecanismo.

En el momento de la confirmación de una transacción, se desechan las páginas de sombras previas y liberar las páginas antiguas.

Las ventajas de este método pasan por la facilidad de recuperación, mientras que las desventajas apuntan al costo de mantener una cantidad de páginas extra que no se mantienen con otras técnicas.

Esquema para una recuperación catastrófica

- Back up Full 1
 - Back up Diferencial 1.1
 - Back up Diferencial 1.2
 - Back up Diferencial 1. n

- Back up Full 2
 - Back up Diferencial 2.1
 - Back up Diferencial 2.2
 - Back up Diferencial 2.n
 - Back up Full m
 - Back up Diferencial m.1
 - Back up Diferencial m.2
 - Back up Diferencial m.n
 - Log transaccional
-

Bibliografía y referencias

- *"Sistemas de Bases de Datos - Conceptos fundamentales" - Elmasti/Navathe - Segunda Edición Adison Wesley Iberoamericana.*
- *"Sistemas de Gestión de Bases de Datos" - Ramakrishnan/Gerke - Tercera edición Ed. McGraw Hill*
- *Wikipedia*
- *"Distributed Databases" - M. Tamer Özsu (University of Alberta)*
- *"Distributed Transaction Processing" - JSTORM (<http://www.jstorm.pe.kr>)*